Arrays

15-3-26

Aim – Write a program to create a 1D, 2D and 3D NumPy array. Perform basic operations like reshaping, slicing, and indexing.Calculate the dot product and cross product of 2 vectors.
Develop a python script to create 2 arrays of the same shape and perform element wise addition, subtraction, multiplication, and division.

Theory - NumPy is a powerful Python library used for numerical computing. It provides support for multidimensional arrays (1D, 2D, 3D) and efficient operations on them. It allows reshaping arrays, slicing, and indexing to access or modify elements easily. NumPy also supports mathematical operations like addition, subtraction, multiplication, and division, as well as advanced computations such as dot product and cross product. It is widely used for scientific computing, data analysis, and machine learning due to its speed and efficiency.

A1.

```
import numpy as np


array_1d = np.array([1, 2, 3, 4, 5])

print("1D Array:")

print(array_1d)


array_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

print("\n2D Array:")

print(array_2d)


array_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

print("\n3D Array:")

print(array_3d)


print("\n--- Reshaping ---")

reshaped_1d = array_1d.reshape(5, 1)

print("1D reshaped to 5x1:")

print(reshaped_1d)


reshaped_2d = array_2d.reshape(1, 9)

print("\n2D reshaped to 1x9:")

print(reshaped_2d)
```

```
print("\n--- Slicing ---")
print("1D array slice [1:4]:", array_1d[1:4])
print("2D array slice [0:2, 1:3]:")
print(array_2d[0:2, 1:3])
print("3D array slice [0, 1, :]:")
print(array_3d[0, 1, :])

print("\n--- Indexing ---")
print("1D array index [2]:", array_1d[2])
print("2D array index [1, 2]:", array_2d[1, 2])
print("3D array index [1, 0, 1]:", array_3d[1, 0, 1])

print("\n--- Dot Product ---")
vector1 = np.array([1, 2, 3])
vector2 = np.array([4, 5, 6])
dot_product = np.dot(vector1, vector2)
print(f"Vector 1: {vector1}")
print(f"Vector 2: {vector2}")
print(f"Dot Product: {dot_product}")

print("\n--- Cross Product ---")
cross_product = np.cross(vector1, vector2)
print(f"Cross Product: {cross_product}")
```

```
1D Array:
[1 2 3 4 5]

2D Array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

3D Array:
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]

--- Reshaping ---
1D reshaped to 5x1:
[[1]
 [2]
 [3]
 [4]
 [5]]
```

```
2D reshaped to 1x9:
[[1 2 3 4 5 6 7 8 9]]

--- Slicing ---
1D array slice [1:4]: [2 3 4]
2D array slice [0:2, 1:3]:
[[2 3]
 [5 6]]
3D array slice [0, 1, :]:
[3 4]

--- Indexing ---
1D array index [2]: 3
2D array index [1, 2]: 6
3D array index [1, 0, 1]: 6

--- Dot Product ---
Vector 1: [1 2 3]
Vector 2: [4 5 6]
Dot Product: 32

--- Cross Product ---
Cross Product: [-3  6 -3]
```

A2.

```
import numpy as np

array1 = np.array([10, 20, 30, 40])
array2 = np.array([2, 4, 5, 8])

addition = array1 + array2
print("Addition:", addition)

subtraction = array1 - array2
print("Subtraction:", subtraction)

multiplication = array1 * array2
print("Multiplication:", multiplication)

division = array1 / array2
print("Division:", division)
```

```
Addition: [12 24 35 48]
Subtraction: [ 8 16 25 32]
Multiplication: [ 20  80 150 320]
Division: [5. 5. 6. 5.]
```

Conclusion

This program demonstrates the numpy library and its various use cases.